HydroGuard: Control Inteligente y Autónomo del Nivel de Líquidos (Fase 2)





PONDERACIÓN: 35

Horas Aproximadas: 30

Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en Ciencias y Sistemas

Índice

Índice **1**
Competencias **2**
Objetivos **3**
General 3
Específicos 3
Descripción / Enunciado **4**
Requisitos de Hardware y Software 5
1. Hardware 5
2. Software 5
Flujo de Trabajo 7
Manejo del broker MQTT 7
Base de datos 8
Entregables **9**
Consideraciones **9**
Cronograma **10**
Rúbrica de Calificación **11**
Penalizaciones **12**
Valores **13**
Referencias **13**

Competencias

- **Desarrolla arquitecturas IoT distribuidas** utilizando microcontroladores (Arduino con módulo WiFi) o dispositivos de borde (Raspberry Pi con WiFi integrado), estableciendo protocolos de comunicación para el intercambio de datos en tiempo real entre sistemas embebidos y plataformas de procesamiento.
- **Implementa sistemas de messaging distribuido** utilizando protocolos MQTT y brokers como Mosquitto en contenedores Docker, diseñando topologías pub/sub que permitan la comunicación asíncrona entre múltiples dispositivos IoT, dashboards web y sistemas de almacenamiento de datos.
- **Diseña dashboards interactivos en tiempo real** empleando tecnologías como React y librerías de visualización como p5.js, creando dashboards que consuman datos de sensores vía API proporcionen controles remotos para la supervisión y operación de sistemas industriales desde cualquier ubicación.
- **Construye sistemas de persistencia de datos** implementando bases de datos relacionales optimizadas para el almacenamiento de series temporales, eventos críticos y logs de operación, diseñando esquemas normalizados que permitan análisis históricos, generación de reportes y trazabilidad completa de operaciones del sistema.
- **Desarrolla mecanismos de control remoto y seguridad distribuida** que permitan la ejecución de comandos críticos (parada de emergencia, cambios de modo operativo, control manual) desde interfaces web, implementando validaciones, autenticación y sistemas de auditoría para garantizar operaciones seguras en entornos industriales.
- **Realiza análisis predictivo y optimización de procesos** utilizando datos históricos almacenados para identificar patrones de comportamiento, calcular métricas de eficiencia del sistema, detectar anomalías operativas y generar alertas proactivas que prevengan fallas o condiciones de riesgo en el sistema de almacenamiento.
- **Integra sistemas de monitoreo híbrido local-remoto** combinando interfaces físicas (LCD, botones) con dashboards web centralizados, asegurando redundancia operativa y manteniendo funcionalidad crítica tanto en conectividad normal como en escenarios de pérdida de comunicación de red.

Objetivos

General

*Se enfoca en la **arquitectura IoT completa**, abarcando desde dispositivos embebidos hasta análisis de datos, con énfasis en soluciones **escalables, seguras y eficientes** para entornos industriales modernos.*

Específicos

- **Implementar la comunicación** de sensores y actuadores mediante un único dispositivo (Arduino con módulo WiFi o Raspberry Pi) utilizando MQTT con Mosquitto en contenedores para la transmisión y recepción de datos en tiempo real.
- **Diseñar e implementar un dashboard interactivo utilizando un framework (React, Angular, Vue, etc) y p5.js** que permita la visualización de datos del sistema y el control de funciones críticas, como la parada de emergencia y el cambio de modo automático/manual de la bomba.
- **Crear una base de datos relacional para almacenar registros históricos del sistema,** incluyendo eventos importantes, mediciones de temperatura y humedad, distancia y tiempos de operación de la bomba, integrando estos datos en el frontend para análisis y monitoreo (datos de la práctica 1).

Descripción / Enunciado

La empresa ficticia **“Químicos Industriales S.A.”**, dedicada al manejo y almacenamiento de líquidos, ha decidido modernizar y automatizar su sistema de monitoreo de silos, con el objetivo de mejorar el control operativo, prevenir accidentes y generar registros históricos confiables. Como paso inicial hacia la digitalización completa de su planta, la empresa requiere evolucionar el prototipo desarrollado en la **Fase 1**, adaptándolo para funcionar como un sistema IoT de monitoreo y control en tiempo real.

El sistema podrá implementarse utilizando ya sea un Arduino con módulo WiFi y sensores conectados, o una Raspberry Pi con sensores y actuadores, para la adquisición de datos (nivel de líquido, temperatura, humedad) y control de dispositivos (bomba, alarma, parada de emergencia).

La comunicación entre el dispositivo central (Arduino con módulo WiFi o Raspberry Pi) y el dashboard se realizará mediante **MQTT con Mosquitto en contenedores**, de modo que dicho dispositivo funcione como **publicador y suscriptor**, enviando datos hacia los usuarios y recibiendo comandos de control en tiempo real. Además, se deberá desarrollar un **dashboard web interactivo**, utilizando tecnologías como React o Angular y p5.js, que permita **visualizar las variables del sistema, controlar la bomba, activar la parada de emergencia y alternar entre modos automático y manual**.

Por último, será necesario implementar una **base de datos relacional** para almacenar registros históricos del sistema, como eventos críticos, niveles de líquido, temperatura, humedad, tiempos de operación de la bomba y paradas del sistema. Una **API** servirá como intermediario entre la base de datos y el registro de datos históricos en el Frontend, permitiendo la consulta y actualización de datos para generar gráficas y reportes históricos.

Con esta implementación, se busca no solo cumplir con los requerimientos operativos inmediatos, sino también establecer una base escalable para la futura digitalización completa de la planta y la integración a un sistema de monitoreo centralizado vía red WiFi.

Requisitos de Hardware y Software

El prototipo físico debe ser el mismo utilizado para la Práctica 1, agregando las siguientes características:

1. Hardware

- **Raspberry Pi (con WiFi integrado) o Arduino (UNO, Mega o similar) con módulo WiFi (ESP8266 o ESP32):** funcionará como nodo central del sistema, encargándose de adquirir los datos de los sensores (nivel de líquido, temperatura del líquido, temperatura y humedad ambiental), procesarlos y distribuirlos mediante MQTT a los suscriptores (dashboard web y otros sistemas). Además, permitirá recibir comandos de control desde el dashboard para accionar la bomba (solo en modo manual), activar la parada de emergencia, alternar entre modos automático y manual, y apagar la alarma sonora.
- **Sensores y dispositivos físicos** Se utilizarán los mismos sensores y dispositivos de la **Fase 1**, incluyendo:
    - Sensor ultrasónico para nivel de líquido.
    - Sonda de temperatura sumergible (DS18B20) para medir la temperatura del líquido.
    - Sensor DHT11 para temperatura y humedad ambiental.
    - Bomba de vaciado controlada por relé.
    - Botón físico de parada de emergencia.
    - Interruptor de apagado automático
    - Buzzer para alertas sonoras.
    - Pantalla LCD 16x2 para visualización local de datos y estado del sistema.

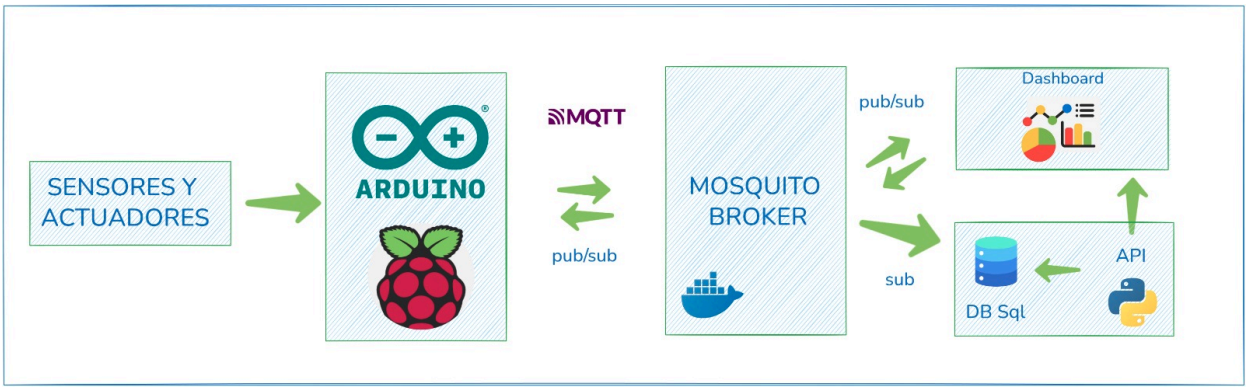
2. Software

- **Broker MQTT con Mosquitto**: e utilizará Mosquitto como **broker MQTT** para manejar la comunicación en tiempo real entre el Arduino o Raspberry Pi (publicador/suscriptor) y el Backend/Frontend, permitiendo el intercambio eficiente de datos de monitoreo y comandos de control. Se implementará mediante contenedores para facilitar su despliegue y portabilidad.
- **Base de datos relacional**: Se utilizará una base de datos relacional (cualquier DMS de su conveniencia) para **almacenar registros históricos** del sistema,

incluyendo niveles de líquido, temperaturas, humedad, Esto permitirá generar reportes y gráficas históricas en el frontend.

- **API con FastAPI o Flask**: Se desarrollará una API que actuará como **puente central entre la base de datos y el frontend**. La API se encargará de enviar registros históricos desde la base de datos hacia el frontend para el modelado de gráficas o histogramas.
- **Frontend Web:** El dashboard se implementará en una PC externa utilizando un framework web moderno (por ejemplo, React)
    - Se debe implementar una vista en tiempo real, en la que se hará uso de **MQTT**, con el fin de tener los cambios al instante y controlar los dispositivos en tiempo real.
    - Obtendrá los datos históricos del sistema a través de la API, lo que simplifica la seguridad y permite manejar autenticación y filtrado de datos, esta será una segunda vista en el dashboard.
    - Esta arquitectura permite separar la lógica de visualización del manejo de mensajes MQTT y de la persistencia de datos.

Flujo de Trabajo



Nota: Se debe utilizar una de las dos plataformas (Arduino + modulo WiFi o Raspberry Pi), queda anulado el uso de las dos plataformas al mismo tiempo.

Manejo del broker MQTT

Para el manejo del broker mqtt se recomienda la siguiente estructura de topics:

- **Estado general del sistema**
  - `silo/sistema/estado` → estado general (`{"modo":"automatico", "emergencia":false, "alarma":false}`)
  - `silo/sistema/evento` → eventos críticos (ej: sobrellenado, fuga, error sensor)
- **Bomba**
  - `silo/bomba/estado` → ON / OFF
  - `silo/bomba/control` → comandos `{"accion":"ON"}` o `{"accion":"OFF"}` (solo en modo manual)

- **Parada de emergencia**
  - `silo/emergencia/control` → `{"accion":"ON"}` activar remotamente la parada de emergencia

- **Modo automático / manual**

    - `silo/modo/estado` → `automatico` / `manual` el estado de trabajo del sistema
    - `silo/modo/control` → cambiar el modo `{"accion":"MANUAL"}` o `{"accion":"AUTOMATICO"}`

- **Alarma sonora**
    - `silo/alarma/estado` → ON / OFF
    - `silo/alarma/control` → `{"accion":"OFF"}` desactivar alarma (solo en modo manual)
- **Sensores**
    - `silo/sensores/nivel` → nivel de líquido en %
    - `silo/sensores/temp_liquido` → temperatura del líquido
    - `silo/sensores/temp_ambiente` → temperatura ambiente
    - `silo/sensores/humedad` → humedad relativa
    - `silo/sensores/tiempo_llenado` → tiempo aproximado hasta el llenado

Base de datos

Se recomiendo el uso de tablas separadas para cada uno de los sensores que se requieren datos históricos, los datos a almacenar son el valor, la unidad, la fecha y hora del registro, sugerencia de tablas:

- nivel de líquido
- temperatura de líquido
- temperatura ambiente
- humedad

Entregables

- Prototipo físico
- Documentación
    - Diagrama de conexión de la Arquitectura.
    - Manual Técnico
- Código Fuente

Consideraciones

- Se valorará la claridad del código, la modularidad y la eficiencia.
- El prototipo profesional y funcional debe ser visiblemente estético.
- Deben cumplirse todos los requisitos.
- La entrega debe realizarse en la fecha establecida.
- El código debe ser subido a un repositorio de Github agregando al auxiliar correspondiente (Auxiliar 1: LuisduPacheco, Auxiliar 2: SmillerMP).
- El repositorio debe nombrarse de la siguiente manera: ACE2_2S25_#Grupo (**Ejemplo: ACE2_2S25_1**) el mismo será utilizado para la Práctica 1, Proyecto 1 y Proyecto 2.
- La detección de copias parciales o totales será motivo de anulación de la práctica con nota 0 de forma automática y se notificará a la Escuela de Ciencias y Sistemas.

Cronograma

El cronograma permite planificar y organizar las diferentes etapas del desarrollo de la práctica

Tarea	Fecha
Asignación de la práctica / Entrega del enunciado	Semana 5 - 23/08/2025
Fecha de entrega	Semana 9 - 19/09/2025
Fecha de calificación	Semana 9 - **20/09/2025**

Rúbrica de Calificación

Área	Puntos Totales	Puntos Obtenidos
CONOCIMIENTO (Subtotal: 75 pts)		
Funcionalidad del Proyecto 70 pts)		
Funcionalidad de todos los sensores y actuadores	5	
Creación de la base de datos y sus tablas	5	
Creación de API para la conexión entre el Backend y Frontend	6	
Subscriber para el manejo de los datos históricos	6	
Comunicación MQTT entre (Arduino o Raspberry Pi) y el broker	4	
Manejo del dashboard para control en tiempo real	12	
Dashboard histórico utilizando la API y la DB	12	
Creación del broker MQTT utilizando contenedores	10	
Gráficas utilizando P5.js	10	
Sub-Total	**70**	
DOCUMENTACIÓN (Subtotal: 5 pts)		
Diagrama de conexión	2	
Manual técnico	3	
Sub-Total	**5**	
HABILIDADES (Subtotal: 25 pts)		
Preguntas relacionadas al proyecto	10	
Maquetación, Encapsulado (15 pts)		

Área	Puntos Totales	Puntos Obtenidos
Estética	7.5	
Diseño	7.5	
Sub-Total	**15**	
TOTAL	**100**	

Penalizaciones

- La entrega tarde tiene una penalización del 100%.
- Las copias parciales o totales tienen una penalización del 100%.
- Commits fuera de tiempo tiene una penalización del 50%.
- No agregar al auxiliar al repositorio tiene una penalización del 30%.
- No utilizar interrupciones en la práctica tiene una penalización del 20%
- No utilizar MQTT para la comunicación entre dispositivos y plataformas 50%
- No presentar un prototipo funcional 30%
- No seguir el diagrama de flujo de trabajo para la arquitectura 40%

Valores

- Responsabilidad: Asumir compromisos y cumplir con tareas asignadas por el equipo de trabajo durante el desarrollo de la práctica.
- Perseverancia: Seguir adelante, incluso cuando hay dificultades, es lo que permite el crecimiento como estudiantes, seguir trabajando en equipo es clave para alcanzar las metas.
- Empatía: Comprender y respetar las opiniones y diferentes aportes de todos los integrantes del equipo de trabajo.

Referencias

- https://p5js.org/
- https://www.raspberrypi.org/
- https://www.mosquitto.org/
- https://roboticsbackend.com/raspberry-pi-arduino-serial-communication/
- https://electronicarych.com/shop/r-pi0w2-r-pi0w2-raspberry-pi-zero-w-2-15076#attr=